Structures

C – Structures

Introduction – Need

There are cases where we need to store multiple attributes of an entity.

It is not necessary that an entity has all the information of one type only.

It can have different attributes of different data types.

For example, an entity Student may have its

name (string),

roll number (int),

marks (float).

To store such type of information regarding an entity student, we have the following approaches:

- Construct individual arrays for storing names, roll numbers, and marks.
- Use a special data structure to store the collection of different data types.

C – Structures

Structure: Definition - A user-defined data type that can be used to group items of possibly different types into a single type.

The **struct** keyword is used to define the structure in the C programming language. The items in the structure are called its **member** and they can be of any valid data type.

Additionally, the values of a structure are **stored in contiguous memory locations**.

Syntax:

```
struct structure_name
{
   data_type member1;
   data_type member2;
   .
   .
   data_type memeberN;
};
```

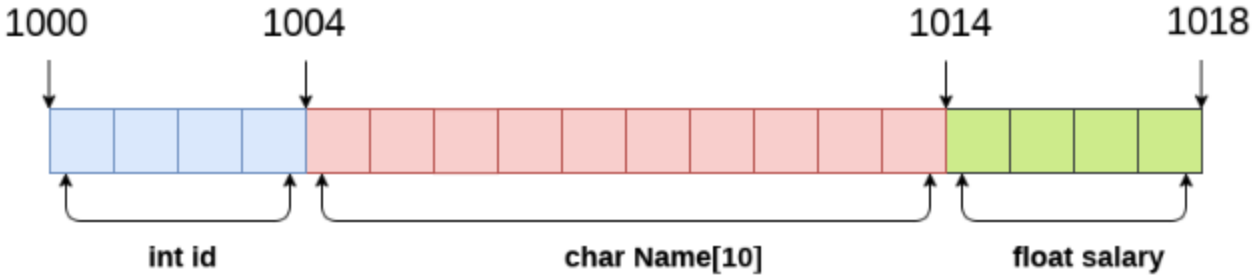
Example:

```
struct employee
{
   int id;
   char name[20];
   float salary;
};
```

C – Structures

Example:

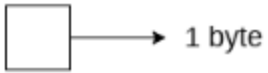
```
struct employee
{
    int id;
    char name[20];
    float salary;
};
```

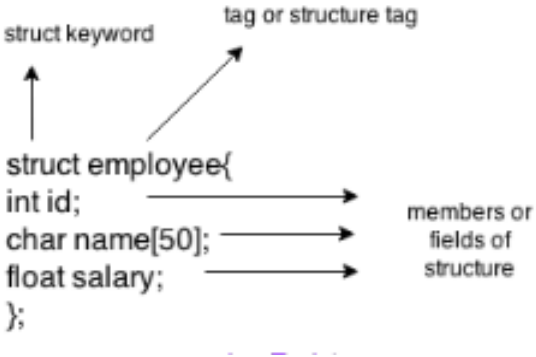



```
struct Employee
{
    int id;
    char Name[10];
    float salary;
} emp;
```

sizeof (emp) = 4 + 10 + 4 = 18 bytes

where;
sizeof (int) = 4 byte
sizeof (char) = 1 byte
sizeof (float) = 4 byte





Declaring structure variable

- We can declare a variable for the structure so that we can access the member of the structure easily. There are two ways to declare structure variable:

1. By struct keyword within main() function
2. By declaring a variable at the time of defining the structure.

Method 1(inside the main() function.)

```
struct employee
{   int id;
    char name[50];
    float salary;
};
void main()
{
 struct employee e1, e2;
   …..
}
```

If number of variables are not fixed, use the 1st approach. It provides you the flexibility to declare the structure variable many times.

Method 2(at the time of defining the structure.)

```
struct employee
{   int id;
    char name[50];
    float salary;
} e1, e2;
```

If no. of variables are fixed, use 2nd approach. It saves your code to declare a variable in main() function.

Note that if you a declare a struct variable in this way, then you cannot initialize it with curly brackets. Instead, the elements need to be assigned individually.

Accessing members of the structure

There are two ways to access structure members:

- By . (member or dot operator)
- By -> (structure pointer operator)

Eg: 1 employee

```
#include<stdio.h>
#include <string.h>
struct employee
{
   int id;
   char name[50];
}e1;  //declaring e1 variable for structure
int main( )
{
   //store first employee information
  e1.id=101;
   //copying string into char array
   strcpy(e1.name, “Sreeram");
   //printing first employee information
   printf( "employee 1 id : %d\n", e1.id);
   printf( "employee 1 name : %s\n", e1.name);
    printf("Size of emp struct: %d", sizeof(struct
                                          employee));
   return 0;
}
```

Output:

employee 1 id : 101
employee 1 name : Sreeram

Accessing members of the structure

Eg: 2 employee

```
#include<stdio.h>
#include <string.h>
struct employee
{   int id;
    char name[50];
    float salary;
}e1,e2;  //declaring e1 and e2 variables for structure
int main( )
{
   //store first employee information
   e1.id=101;
   strcpy(e1.name, “Namita");
   //copying string into char array
   e1.salary=56000;
   e2.id=102;
   strcpy(e2.name, “Nandita");
   e2.salary=126000;
   //printing first employee information
   printf( "employee 1 id : %d\n", e1.id);
   printf( "employee 1 name : %s\n", e1.name);
  printf( "employee 1 salary : %f\n", e1.salary);

 //printing second employee information
    printf( "employee 2 id : %d\n", e2.id);
    printf( "employee 2 name : %s\n", e2.name);
    printf( "employee 2 salary : %f\n", e2.salary);
    return 0;
 }
```

employee 1 id : 101
employee 1 name : Namita
employee 1 salary : 56000.000000
employee 2 id : 102
employee 2 name : Nandita
employee 2 salary : 126000.000000

Additional Question

Create a structure for a **book** described
by **title** (string), **author** (string), **price** (double), **number of pages** (integer), etc.

```
struct book
{
    char title[50];
    char author[50];
    double price;
    int pages;
} book1;
```

Structure Initialization

The **initialization** of a struct variable is done by placing the value of each element inside curly brackets.

Example

struct book book1= {"Learn C", "Dennis Ritchie", 675.50, 325};

```
struct book
{
    char title[50];
    char author[50];
    double price;
    int pages;
};
```

Copying Structures

```
struct book
{
    char title[50];
    char author[50];
    double price;
    int pages;
};
```

The **assignment (=) operator** can be used to copy a structure directly.
You can also use the assignment operator (=) to assign the value of the member of one structure to another.
Let's have two struct book variables, **book1** and **book2**.
The variable **book1** is initialized with declaration, and we wish to assign the same values of its elements to that of **book2**.

```
struct book book1 = {"Learn C", "Dennis Ritchie", 675.50, 325}, book2;
strcpy(book2.title, book1.title);
strcpy(book2.author, book1.author);
book2.price = book1.price;
book2.pages = book1.pages;
```

Copying Structures

```
#include <stdio.h>
#include <string.h>
struct book
{
   char title[10];
   char author[20];
   double price;
   int pages;
};
```

```
int main()
{
  struct book book1 = {"Learn C", "Dennis
                       Ritchie", 675.50, 325}, book2;
  book2 = book1;
  printf("Title: %s \n", book2.title);
  printf("Author: %s \n", book2.author);
  printf("Price: %lf \n", book1.price);
  printf("Pages: %d \n", book1.pages);
  printf("Size of book struct: %d", sizeof(struct
                                          book));
  return 0;
}
```

Structures as Function Arguments

You can pass a structure as a function argument in the same way as you pass any other variable or pointer.

Pointers to Structures

You can define pointers to structures in the same way as you define pointers to any other variable.

Declaration of Pointer to a Structure

You can declare a pointer to a structure (or structure pointer) as follows −

```
struct Books *struct_pointer;
```

Initialization of Pointer to a Structure

You can store the address of a structure variable in the above pointer variable **struct_pointer**. To find the address of a structure variable, place the '&' operator before the structure's name as follows −

```
struct_pointer = & book1;
```

Pointers to Structures

To store the address of a struct variable in a struct pointer variable;

```
struct book
{
  char title[10];
  char author[20];
  double price;
  int pages;
};

struct book book1 = {"Learn C", "Dennis
  Ritchie", 675.50, 325}, struct book *strptr;
```

Accessing Members Using Pointer to a Structure

To access the members of a structure using a pointer to that structure, you must use the → **operator** as follows −

```
struct_pointer->title;
```

C defines the → symbol to be used with struct pointer as the **indirection operator** (also called **struct dereference operator**). It helps to access the elements of the struct variable to which the pointer reference to.

Pointers to Structures

Note: The dot (.) operator is used to access the struct elements via the **struct** variable. To access the elements via its pointer, we must use the **indirection** (->) operator

In this example, **strptr** is a pointer to **struct book book1** variable.
Hence, **strrptr→title** returns the title, just like **book1.title** does.

```
#include <stdio.h>
#include <string.h>
struct book
{
  char title[10];
  char author[20];
  double price;
  int pages;
};
int main ()
{
   struct book book1 = {"Learn C", "Dennis
                                Ritchie", 675.50, 325};
   struct book *strptr;
   strptr = &book1;
   printf("Title: %s \n", strptr -> title);
   printf("Author: %s \n", strptr -> author);
   printf("Price: %lf \n", strptr -> price);
   printf("Pages: %d \n", strptr -> pages);
   return 0;
}
```

Array of Structures

An array whose elements are of type structure is called array of structure. It is generally useful when we need multiple structure variables in our program.

Need for Array of Structures

Suppose we have 50 employees and we need to store the data of 50 employees. So for that, we need to define 50 variables of struct Employee type and store the data within that. However, declaring and handling the 50 variables is not an easy task. Let's imagine a bigger scenario, like 1000 employees.

Then,

struct Employee emp1, emp2, emp3, ... emp1000;

For that, we can define an array whose data type will be struct Employee soo that will be easily manageable.

struct *structure_name* *array_name* [number_of_elements];

Array of Structures

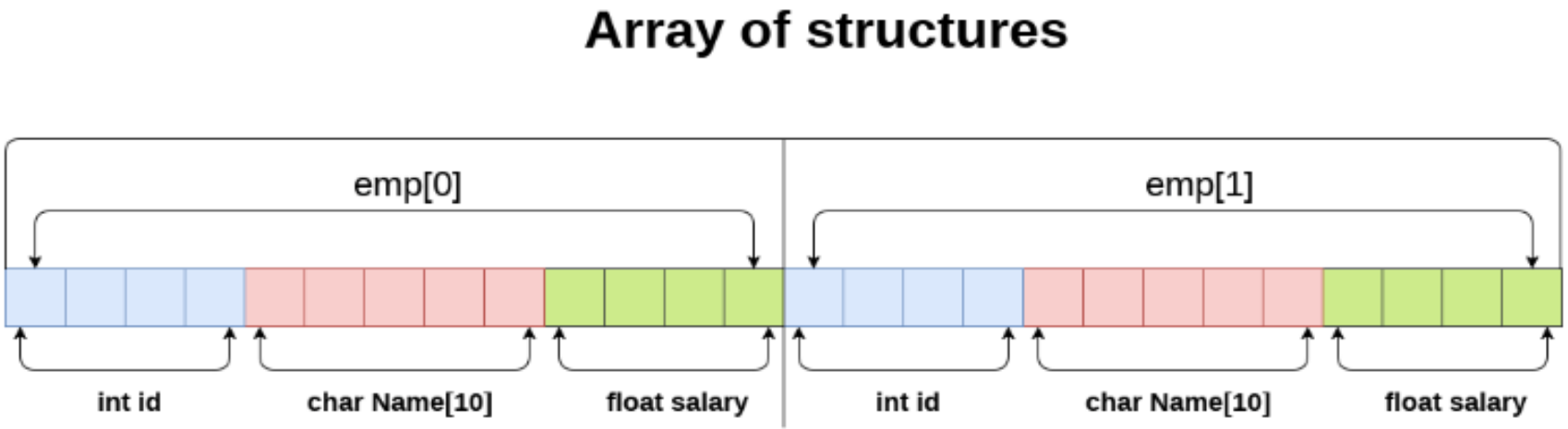
Initialization of Array of Structures

We can initialize the array of structures in the following ways:

struct *structure_name array_name* [number_of_elements] =

```
{
   {element1_value1, element1_value2, ....},
   {element2_value1, element2_value2, ....},
     ......
     ......
};
```

Memory Allocation for Array of Structures




```
struct employee
{
    int id;
    char name[5];
    float salary;
};
struct employee emp[2];
```

sizeof (emp) = 4 + 5 + 4 = 13 bytes

sizeof (emp[2]) = 26 bytes

Array of Structures

```
#include<stdio.h>
#include <string.h>
struct student
{
  int rollno;
  char name[10];
};
int main(){
int i;
struct student st[5];
printf("Enter Records of 5 students");
for(i=0;i<5;i++)
{
   printf("\nEnter Rollno:");
   scanf("%d",&st[i].rollno);
   printf("\nEnter Name:");

scanf("%s",&st[i].name);
}
printf("\nStudent Information List:");
for(i=0;i<5;i++)
{
  printf("\nRollno:%d, Name:%s",st[i].rollno,st[i].name);
}
 return 0;
}
```

Array within a Structure

An array can be declared inside a structure as a member when we need to store multiple members of the same type.

For example, suppose we have a student and we need to store the data of his/her marks for different subjects. So for that, we need to define a structure of type student and store the data within that. However, declaring and handling different variables for each subject is not an easy task.

```
struct Employee { int sub1, sub2, sub3, sub4, sub5;
                  };
```

For that, we can define an array within the structure whose data type will be int so that will be easily manageable.

Syntax to Declare Array Within Structure

```
struct StructureName { // Other members
                       dataType arrayName[arraySize];
                     };
```

Note: *It is recommended to define the array as the last member function so that if it overflows, it does not overwrite all the other members.*

Accessing Elements of an Array within a Structure

We can access the elements of the array within the structure using the dot (.) operator along with the array index inside array subscript operator.

```
structureName.arrayName[index]
```

Array within a Structure

```
#include<stdio.h>
struct student
{
  char name[20];
  int roll;
  float marks[5]; /* This is array within structure */
};
int main()
{
  struct student s;
  int i;
  float sum=0, p;
  printf("Enter name and roll number of students:\n");
  scanf("%s%d",s.name,&s.roll);
  printf("\nEnter marks obtained in five different
                    subject\n");
for(i=0;i< 5;i++)
  {
     printf("Enter marks:\n");
     scanf("%f", &s.marks[i]);
     sum = sum + s.marks[i];
  }
   p = sum/5;
   printf(“Student records and percentage is:\n”);
   printf("Name : %s\n", s.name);
   printf("Roll Number : %d\n", s.roll);
   printf("Percentage obtained is: %f", p);
   return 0;
}
```

Advantages of Structures

Advantages:

1. **Heterogeneous collection of data items:** structure allows us to create user defined data-type which can store items with different data types.
2. **Reduced complexity:**
   If we need to create a record of five students with roll number, name, marks and address, then without structure, we need to use Five integer type variables for storing roll number of five students, five string variables for storing name of five students, five floating type variables for storing marks, and again five string variables for storing address of five students, which is very difficult to manage. The use of a structure and an array or structure reduces this complexity.
3. **Increased productivity:** structure in C eliminates lots of burden while dealing with records which contain heterogeneous data items, thereby increasing productivity.

Advantages of structure

4. **Maintainability of code:** using structure, we represent complex records by using a single name, which makes code maintainable.
5. **Enhanced code readability:** code readability is crucial for larger projects. Using structure code looks user friendly which in turn helps to maintain the project.
6. **Suitable for some mathematical operations:** by using basic data-types we can not represent complex numbers, distances (feet-inch system), times (hour-minute-second system). Mathematical operations like adding complex numbers, or subtracting them; adding two distances in a feet-inch system; finding the difference between two time periods in an hour-minute-second system can be done effectively using the concept of structure.

Disadvantages of structure

1. **Can increase complexity of the program:** excessive use of structures in C increases complexity of program and program becomes difficult to manage.
2. **Slower:** excessive use of structures makes programs slower due to storage overhead required for all data.
3. **Hard to manage:** change in one structure variable forces change in many places in the program which makes programmer difficult to maintain and debug if there is excessive use of structure and pointer.
4. **Difficult error tracking:** excessive use of structures in C increases complexity thereby making it difficult for error tracking.
5. **A steep learning curve:** beginners find it difficult to use structure in C thereby making a slow learning curve.
6. **Page faults:** using a lot of pointer dereferences can be a performance hit in structure due to additional memory fetch and locality of references, which may lead to page faults.

Uses of Structure in C

Uses of Structure in C

- The structure can be used to define the custom data types that can be used to create some complex data types such as dates, time, complex numbers, etc. which are not present in the language.
- It can also be used in data organization where a large amount of data can be stored in different fields.
- Structures are used to create data structures such as trees, linked lists, etc.
- They can also be used for returning multiple values from a function.

Limitations of Structure in C

1. **Higher Memory Consumption:** It is due to structure padding.
2. **No Data Hiding:** C Structures do not permit data hiding. Structure members can be accessed by any function, anywhere in the scope of the structure.
3. **Functions inside Structure:** C structures do not permit functions inside the structure so we cannot provide the associated functions.
4. **Static Members:** C Structure cannot have static members inside its body.
5. **Construction creation in Structure:** Structures in C cannot have a constructor inside Structures.

Differentiate between Array and Structure

Array	Structure
Array is collection of homogeneous elements i.e. of similar data types.	Structure is collection of heterogeneous elements i.e. of different data types.
Array uses index or subscript for accessing array elements.	Structure uses dot (.) operator for accessing structure elements.
Accessing array elements takes less time.	Accessing structure elements takes more time.

Array can be declared as:
`int a[100];`
	Structure can be declared as:
	`struct book`
	`{`
	`  int pages;`
	`  float price;`
	`};`

Union

*The Union is a **user-defined data type** in C language that can contain elements of the different data types just like structure. But unlike structures, **all the members in the C union are stored in the same memory location**. Due to this, only one member can store data at the given instance.*

Syntax:

union *union_name*
{
datatype member1;
datatype member2;
...
};

Note:
We only declare the template of the union, i.e., we only declare the members' names and data types along with the name of the union. **No memory is allocated to the union in the declaration.**

Size of Union

The size of the union will always be equal to the size of the largest member of the array. All the less-sized elements can store the data in the same space without any overflow.

Important - **only one member can contain some value at a given instance of time.**

Structures Vs Union

Structure	Union
The size of the structure is equal to or greater than the total size of all of its members.	The size of the union is the size of its largest member.
The structure can contain data in multiple members at the same time.	Only one member can contain data at the same time.
It is declared using the struct keyword.	It is declared using the union keyword.